

Design Note: Claude Code Sub-Agents

30-minute enablement for Applied AI engineers

Learning objectives

- **Explain** the orchestrator/sub-agent mental model to a customer, and articulate why role specialization beats one long-context pass: focused, role-specific reasoning; parallel execution; and task-scoped isolation with structured results.
- **Build** a working sub-agent workflow in Claude Code - define specialist roles, spawn them via the Task tool, synthesize results through the orchestrator - and templize it in CLAUDE.md so it is repeatable across an engineering team.
- **Qualify** where sub-agents are the right tool (multi-step workflows with clear delegation points, parallelizable domains, failure isolation) and where they add friction (simple linear tasks, interdependent shared state, latency-sensitive paths, cost/rate-limit/context overhead).

How the session is scoped

In scope	Out of scope
The orchestrator/sub-agent pattern	Multi-orchestrator and agent-to-agent handoff architectures
Role specialization - Logic Reviewer / Test Writer / Security Auditor versus one long-context reviewer	Prompt-engineering depth for tuning individual specialists
Templatizing - CLAUDE.md locks roles, report format, and guardrails	CI/CD wiring and MCP / external-tool integrations
When/when not to use sub-agents	Pricing, rate-limit, and benchmark deep-dives

Why this demo (PR review with 3 specialists) is right for the field

- **Zero-setup and repeatable:** clone the repo, start Claude Code, type /review. The bundled sample_pr.diff - a realistic Flask + Stripe payments service with bugs planted in every specialist's domain guarantees meaningful findings on every run. No customer data, environment prep, or API plumbing.
- **Makes specialization visceral:** the Logic Reviewer catches a Stripe amount-unit mismatch and a leaked DB connection, the Security Auditor a hardcoded live key and SQL injection, the Test Writer the untested /refund endpoint. This is concrete proof that one reviewer holding all three domains misses things.
- **Quantifiable parallelism:** the three specialists run in parallel via the Task tool, so the audience watches 3× the review coverage land in roughly the wall-clock time of one pass (~12 min sequential → ~5 min).
- **Pivots to the customer's code live:** /review accepts any diff - git diff or gh pr diff, so the AAI engineer can rerun the same workflow on the prospect's actual PR in the same meeting.
- **Leaves an adoptable asset:** CLAUDE.md locks in the roles, workflow, and report format; adding a specialist is a copy-paste edit. The demo repo is the leave-behind - a customer can adapt it to their team the same day.

Signal that the session worked

Session: at the close, ask “where in your current customer pipeline would you use a role-specialized subagent workflow, and where wouldn't you?” A passing answer names a real account, assigns concrete specialist roles, explains why specialization beats a single long-context call, and identifies at least one constraint unprompted.

Activation: the first confirmed sub-agent workflow run in a customer's repo is the milestone, logged when the customer shares their committed CLAUDE.md or a first review report. No run = no activation, regardless of how well the session went.

30-day: time saved per task with sub-agents (benchmarked at session close, revisited at 30 days) and the number of teams with CLAUDE.md committed to their repo — both feed the success story library